Autonomous Exploration, Mapping, and Localization

Dennis Johnson, Annika Trainer, Myles Leatherman

I. Introduction

The goal of this lab is to apply concepts from previous labs to build a complete navigation system for a TurtleBot operating in an unknown maze. Using SLAM Toolbox, the TurtleBot must explore the environment, construct a map, avoid obstacles, and autonomously choose where to move next based on frontiers, regions at the boundary between known free space and unknown space. Once mapping is complete, the robot must navigate back to its starting position. Finally, given only a saved map and LiDAR data, the robot must localize itself using AMCL and drive to a manually selected location, thereby solving the kidnapped robot problem.

II. Methodology

A. Phase 1: Autonomous Exploration and Mapping

Our Phase 1 system combines SLAM Toolbox, a custom Explorer node, a PathPlanner node, and a Controller node to fully automate map creation. SLAM Toolbox runs continuously and publishes an up-to-date occupancy grid on `/map`. The Explorer subscribes to this topic, converts the map data into a 2D array, and performs two key preprocessing steps: C-space inflation and risk-map generation.

For C-space dilation, obstacles are thickened using OpenCV-based dilation with local filtering rules that distinguish noise, unknown space, and true obstacles. An inflation radius of three cells ensures that any pose planned in C-space is collision-free (or close to it) with respect to the TurtleBot's footprint. This inflated C-space map is then passed to `calc_risk_map()`, where an OpenCV Euclidean distance transform computes the distance from each free cell to the nearest obstacle. Distances are mapped to risk values, producing a gradient away from walls; this risk map is later used to bias A* paths away from obstacles. Both the C-space map and the risk map are periodically updated as new SLAM data arrive, published for RViz visualization, and cached so the Explorer and PathPlanner can operate on the latest information.

To identify unexplored regions, the Explorer computes a frontier mask directly from the raw SLAM map. A cell is considered a frontier if it is free and at least one of its four-connected neighbors is unknown. Raw frontier cells are clustered into blobs using dilation and connected-components analysis, yielding contiguous frontier regions instead of scattered cells. A centroid is then computed for each blob.

Because centroids can lie in unsafe or unknown space, they are shifted into the C-space using a helper function `shift_to_known()`, which performs a bounded breadth-first search (BFS) from the centroid. Candidate cells must be walkable and within the known C-space; among these, the cell with the lowest risk is selected. These final target grid cells are stored in a persistent global list that is pruned as frontiers are explored and disappear.

Whenever a new map is received, the Explorer obtains the robot pose using a TF lookup, converts it to the nearest grid cell, and triggers the frontier detection pipeline. If valid frontier targets exist, the Explorer computes their distances to the robot in grid coordinates and chooses the closest one, subject to a lower bound that prevents selecting frontiers essentially beneath the robot. If a near-frontier remains too close for multiple updates, it is pruned from the global list. The selected frontier is converted to a `PoseStamped` and published on `/move_base_simple/goal`, which triggers the PathPlanner.

In the PathPlanner node, each incoming goal from `/move_base_simple/goal` is paired with the robot's current pose to define a start–goal pair. The start pose is obtained via TF from `base_link` into the `map` frame, and both start and goal positions are converted into grid coordinates in C-space. If the start cell is not walkable (for example, if the robot is slightly inside an inflated obstacle due to discretization), the planner calls `find_nearest_walkable_cell()` to run a local breadth-first search in grid space until it finds the closest valid C-space cell, which is then used as the start cell. This only ever becomes important in Phase 2, but it is worth noting here because it guarantees that planning can still succeed even if discretization and inflation temporarily place the robot in an unsafe grid cell.

The planner then runs a risk-aware A* search over eight-connected neighbors. The step cost combines Euclidean distance with a risk penalty derived from the risk map, while the heuristic term is primarily Euclidean distance to the goal plus a small heuristic risk term. After A* finds a path, the resulting grid-cell sequence is simplified in `optimize_path()` by removing intermediate cells that lie on the same straight-line direction segments. The final grid path is converted into a sequence of `PoseStamped` waypoints in the `map` frame, each with a position derived

from the grid cell and an orientation aligned with the direction of travel. This full path is published as a `Path` message on `/nav_path` and sent as a `FollowPath` action goal to the Controller.

The Controller node implements the `FollowPath` action server and handles low-level locomotion. When it receives a `FollowPath` goal, it iterates through the waypoints and calls `go_to()` for each pose. Inside `go_to()`, the target pose is transformed into the `odom` frame, and the Controller uses two motion primitives: `smooth_rotate()` to align the robot's heading with the waypoint, and `smooth_drive()` to move forward. Both use trapezoidal velocity profiles parameterized by elapsed time, with acceleration and deceleration phases that respect maximum linear and angular velocities. `smooth_drive()` applies a proportional heading correction based on the difference between current yaw and desired heading. During execution, the Controller publishes diagnostic error signals (distance error and yaw) on debugging topics.

In parallel, the Controller continuously monitors `LaserScan` data in `scan_callback()` and implements an emergency-stop (E-stop) layer. The scan is processed to find the minimum distance in a front-facing angular window and in a rear-facing window. If an obstacle is detected closer than a configurable threshold in front of the robot, the Controller sets an internal E-stop flag, immediately zeros commanded velocity, and backs up slightly while preventing further forward motion. If this E-stop occurs while a `FollowPath` goal is active, the Controller marks the goal as aborted with an "E-STOP" message. The PathPlanner listens to the `FollowPath` result and, if E-stop replanning is enabled, will replan up to a specified maximum number of times toward the same goal using the updated map and pose. The E-stop flag is cleared when the scan shows that both front and back distances are safe and roughly symmetric, indicating the robot is no longer too close to an obstacle and is in a safer location.

Exploration continues through this loop of frontier selection, planning, and path execution until no valid frontier centers remain in the global list and the frontier detector can no longer find new frontiers in the map. At this point, the Explorer treats the map as complete and triggers the map-saving routine. It uses the Nav2 `/map_saver/save_map` service to store the final occupancy grid and metadata to a directory under a chosen name. Once the map is saved successfully, the system marks exploration as finished and transitions to Phase 1.5.

B. Phase 1.5: Return Home

In Phase 1.5, the Explorer uses the home position, defined as the first valid pose it received from TF when SLAM started and the robot began exploring. Before commanding a return, the Explorer verifies that this home pose lies in safe C-space. If so, it publishes a `PoseStamped` goal on `/explorer/go_home`. If the original home cell is no longer safe (for example, due to subsequent obstacle inflation), the Explorer converts the home pose into grid coordinates and calls `find_nearest_walkable_cell()` to locate the nearest valid C-space cell, then converts that back into a world position and uses that as the return goal.

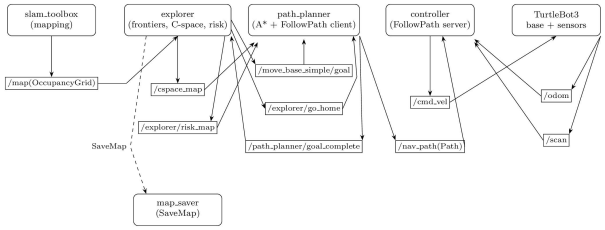


Fig. 1. System diagram showing complete Phase 1 and Phase 1.5 logic and structure.

This go-home goal is processed by the PathPlanner using the same A* pipeline as in Phase 1, but with path trimming disabled so the robot follows the full path back. The Controller executes the return path through the `FollowPath` action exactly as before, with the same E-stop protections. When the Controller reports successful completion, the PathPlanner publishes a goal-complete message, and the Explorer marks the mission as fully complete, ending Phase 1.5. Overall, the full Phase 1 system is similar to our PDR, with obvious refinements and additions to the internal logic.

C. Phase 2: AMCL and the Kidnapped Robot Problem

To complete Phase 2 of the lab, the kidnapped robot problem, we needed to efficiently integrate AMCL into our system. AMCL, or Adaptive Monte Carlo Localization, uses probabilistic localization to estimate the robot's position and orientation based on LiDAR data. We began by creating a new launch file based on our SLAM simulation launch file. In this version, we removed the SLAM node and added the AMCL node, making sure the scan topic was properly remapped to `/scan`. After generating a map with SLAM, we needed to save it in the `go_home()` method. To do this, we added a map server node to the launch file, configured it to save maps into a folder called `maps`, and specified that the map should be saved as `explored_map.yaml`. In the Explorer code, we then added a function to save the map by creating a client for the `/map_saver/save_map` service and sending a `SaveMap.Request()` with the appropriate file path. We also changed the durability of the `/map` topic in RViz from volatile to transient local so the map would display correctly.

To handle localization after the robot was kidnapped, we created another node called `blowup`, which manages the

robot's initial pose and re-localization. This node creates clients for `reinitialize_global_localization` and `request_nomotion_update`. It sends an empty request to `reinitialize_global_localization` once to reset the pose estimate, and then sends 10,000 empty requests to `request_nomotion_update`, with 0.15-second pauses between them to avoid overloading the system. This allows AMCL to converge on the robot's correct location on the map. After adding this node to the launch file, we were able to successfully solve the kidnapping problem. Once again, the Phase 2 system is similar to our PDR. In this case, the system is a little bit simpler, as we simplified the logic with the AMCL service calls.

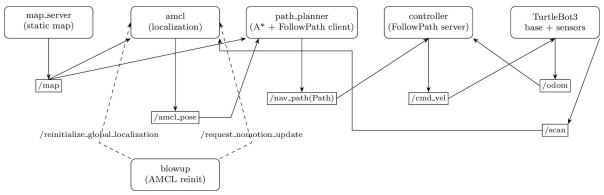


Fig. 2. System diagram showing complete Phase 2 AMCL localization logic and structure.

III. Results

Figure 3 shows the robot exploring the environment in simulation. The SLAM map is displayed in RViz with frontiers rendered in blue, the planned path as a light green line, and the robot pose as a red arrow. The robot incrementally builds the map while repeatedly selecting frontier goals.

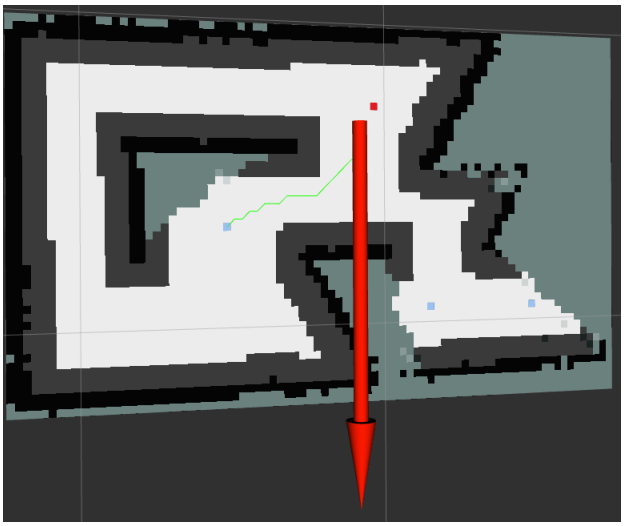


Fig. 3. RViz SLAM map in simulation. Frontiers (blue), planned path (light green), and robot pose (red arrow) during exploration.

In AMCL experiments, the initial pose estimate uses a very large covariance, represented as a large uncertainty ellipse around the robot (Fig. 4). This reflects the uncertainty immediately after calling `/reinitialize_global_localization`.

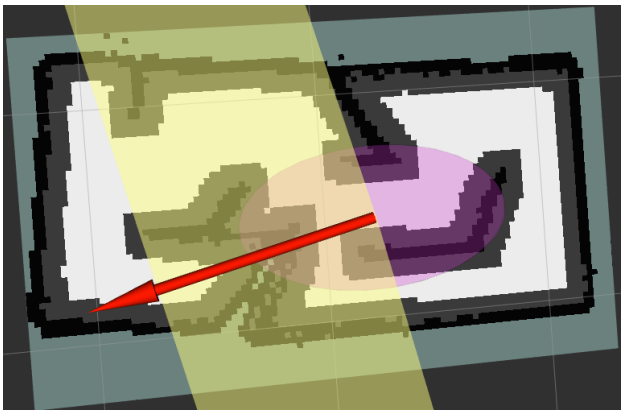


Fig. 4. Initial AMCL pose with large covariance after `reinitialize_global_localization`.

As `/request_nomotion_update` is called repeatedly, AMCL refines its belief, and the covariance decreases as particles converge to the correct pose (Fig. 5).



Fig. 5. Covariance shrinking over time as AMCL converges using continuous `request_nomotion_update` calls.

Once localization is stable, a 2D goal pose is sent in RViz, and the robot follows a planned path to reach the target while AMCL continues to refine the estimate (Fig. 6).

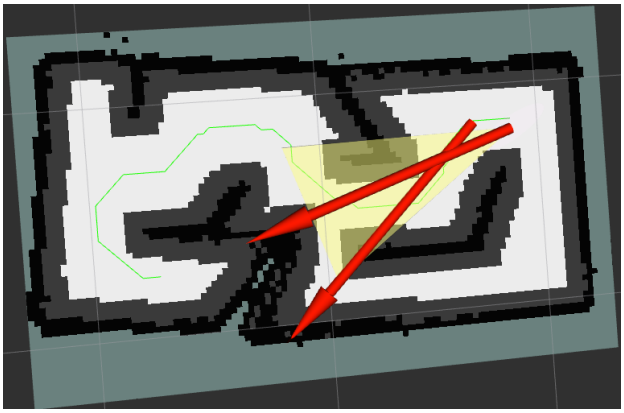


Fig. 6. Robot following a planned path to a goal using AMCL-based localization.

On the physical robot, SLAM produces a final occupancy map of the maze (Fig. 7). This map is captured, saved, and later reused by AMCL for Phase 2 kidnapping experiments.

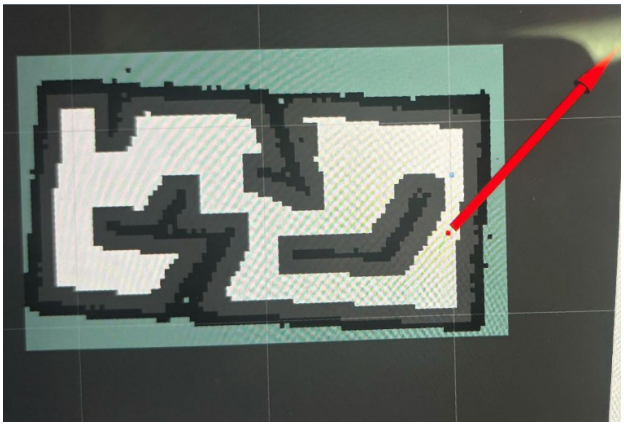


Fig. 7. Final SLAM map captured on the physical TurtleBot and saved for later AMCL use.

IV. Discussion

A. Phase 1: Exploration and Risk-Aware Planning

The primary goal of this lab was to integrate SLAM, autonomous exploration, path planning, and closed-loop control into a single pipeline capable of mapping an unknown environment and returning home reliably. Our Phase 1 and Phase 1.5 implementation reflects this objective: we designed a modular, multi-node architecture where each component performs a focused task: SLAM for mapping, an Explorer for frontier detection, a PathPlanner for safe navigation, and a Controller for low-level motion. This separation of responsibility allowed us to debug and evaluate each stage independently, and our findings supported the need for clear boundaries between perception, planning, and control.

We chose this architecture because it mirrors real robotic navigation systems and provides flexibility during development. Using SLAM Toolbox as the backbone gave us a consistent, incremental occupancy grid and a stable transform tree, which was essential for exploration. Working with the live SLAM map, our Explorer generated frontier regions, corrected unsafe centroids, and selected the next best information-gathering goal. This approach was effective because the SLAM output is noisy and dynamic; frontier clustering plus centroid shifting let us avoid sending the robot to unreachable or unsafe spots. Our testing showed that relying on raw frontier cells without this filtering often resulted in the robot choosing targets too close to walls or obstacles, which justified the additional preprocessing.

The risk map was another design choice shaped by our findings. By running a Euclidean distance transform over the C-space–inflated map, we produced a gradient that encouraged the A* planner to favor paths further from obstacles. This dramatically reduced scraping behavior near walls—an issue that appeared early in testing when the robot attempted to squeeze through narrow regions. Adding risk-aware path costs made the robot's motion more reliable, especially in cluttered sections of the maze.

An important part of our debugging strategy was isolating problems to individual nodes. For example, early failures often came from the Controller receiving unsafe starting poses or the planner sending paths through inflated obstacles. By validating transforms, verifying grid conversions, and visualizing each intermediate artifact in RViz (C-space map, risk map, frontier mask, and A* output), we could identify whether issues originated in SLAM updates, frontier detection, or path generation. This step-by-step debugging process proved more efficient than trying to diagnose the system holistically. Parameter tuning also played a significant role: we adjusted C-space inflation radius, frontier dilation, clustering thresholds, and A* cost weights to balance safety and exploration speed. Increasing the inflation radius from one, to two, to three cells greatly improved reliability in narrow passages. Similarly, weighting the risk term in A* helped the robot maintain smoother, less erratic paths.

B. Phase 1.5

Finally, Phase 1.5 tested the completeness of our architecture. Once exploration ended, we used the saved map and the stored home pose to navigate back. This validated the consistency of our transform handling, map saving, and path planning pipeline. We added a fallback mechanism for unsafe home positions (finding the nearest C-space–valid cell), which proved necessary as SLAM updates sometimes changed obstacle inflation over time.

Looking forward, the main improvements we would make include reducing TF message load to avoid occasional drops, integrating a more robust recovery behavior when frontiers fail repeatedly, and refining the Controller to use pure pursuit instead of separate rotate-then-drive primitives. Despite these

limitations, our system demonstrated a coherent exploration pipeline and effectively met the lab's goal: using SLAM to autonomously map an unknown environment and reliably return to the starting location.

C. Phase 2

For the architectural decisions in Phase 2, we chose to create a separate launch file for the AMCL portion of the lab, as running AMCL and SLAM together can lead to conflicts and unstable behavior. We also created a separate node for the methods used in Phase 2 to simplify debugging; separating the logic made it easier to identify and isolate issues as they arose. For localization, we used the `reinitialize_global_localization` service because it provides an initial pose estimate with a deliberately large covariance, allowing the robot to explore and converge on its true position. We paired this with the `request_nomotion_update` service, which, when called repeatedly, reduces the covariance until AMCL becomes confident in the robot's exact pose. This approach consistently produced accurate results, so we decided to use it.

We encountered two main debugging issues related to RViz configuration. When switching from SLAM to AMCL, several RViz settings changed automatically. The first issue was that the `/map` topic would not appear in RViz even though it was being published correctly. After checking the RViz display settings, we discovered that the durability policy for the `/map` topic had been switched from transient local to volatile. Changing it back allowed our SLAM-generated map to display correctly. The second issue was that the `path_planner` node was not receiving the 2D goal poses we set in RViz. We found that the 2D goal tool had been reconfigured to publish to `/map_update` instead of `/move_base_simple/goal`. After correcting the topic, the `path_planner` node was once again able to receive goal poses successfully.

Overall, the AMCL setup demonstrated that our map-saving, localization, and planning components worked together as intended. The kidnapped robot problem was solved by leveraging the saved SLAM map, AMCL's particle filter, and service-based reinitialization, followed by the same A* and control stack used in Phase 1.

V. Conclusion

Through this lab, we gained a deeper understanding of how perception, planning, and control must operate together for a robot to explore an unknown environment and return home autonomously. By building a modular architecture that used SLAM for mapping, frontier-based exploration for goal generation, risk-aware A* for safe navigation, and closed-loop control for motion, we learned how each subsystem depends on accurate transforms, reliable map data, and careful parameter tuning. Debugging these components individually and validating their interactions in RViz taught us the importance of isolating failures. We did this by visualizing intermediate representations such as frontier masks, inflated maps, planned paths, and, most importantly, debug statements.

In Phase 2, working with AMCL reinforced how localization uncertainty, covariance tuning, and RViz configuration directly impact system behavior. The combination of `reinitialize_global_localization` and repeated `request_nomotion_update` calls provided a practical method for robust re-localization on a static map. Overall, this lab demonstrated the complexity and interdependence of a full robotic navigation pipeline in ROS 2, and gave us practical experience building, tuning, and troubleshooting a system capable of exploring, mapping, localizing, and navigating in real time.

References

- SLAM Toolbox Documentation
- AMCL ROS 2 Package Documentation
- OpenCV Documentation